

**LAPORAN MINI PROJECT UJIAN AKHIR SEMESTER
(UAS)**

**MATA KULIAH: COMMUNICATION PROTOCOL
(ComP2)**

INTEGRASI PROTOKOL, KEANDALAN, DAN OBSERVABILITAS PADA
PIPELINE DATA INGESTION BERBASIS REST API



Nama : Anggi Debia Zahra

NIM : 25120500005

Program Studi : Sains Data

Dosen Pengampu :

Haiqal Shiddiq, S.kom., M.T.

BAB 1

PENDAHULUAN DAN USE CASE

1.1 Latar belakang

Dalam siklus kerja Sains Data, tantangan terbesar sebelum melakukan analisis atau pemodelan *Machine Learning* adalah memastikan kualitas data mentah (*raw data*) yang dikirimkan dari aplikasi sumber. Istilah "*Garbage In, Garbage Out*" menunjukkan bahwa data yang kotor, tidak lengkap, atau cacat struktur akan langsung merusak akurasi model prediktif. Seringkali, data dikirim tanpa adanya validasi di gerbang pertama jaringan, sehingga mempersulit pelacakan sumber *error* ketika pipa data (*data pipeline*) mengalami gangguan.

1.2 Tujuan Mini Project

- Membangun gerbang penyerapan data (*Ingestion API*) otomatis yang handal untuk menerima data transaksi penjualan.
- Mengimplementasikan validasi skema data (*schema validation*) secara ketat di tingkat protokol komunikasi untuk menyaring data cacat secara *real-time*.
- Menerapkan mekanisme observabilitas tingkat tinggi menggunakan kode status HTTP terstandarisasi dan pelacak *Request ID* unik untuk mempermudah proses audit dan pencarian gangguan (*troubleshooting*).

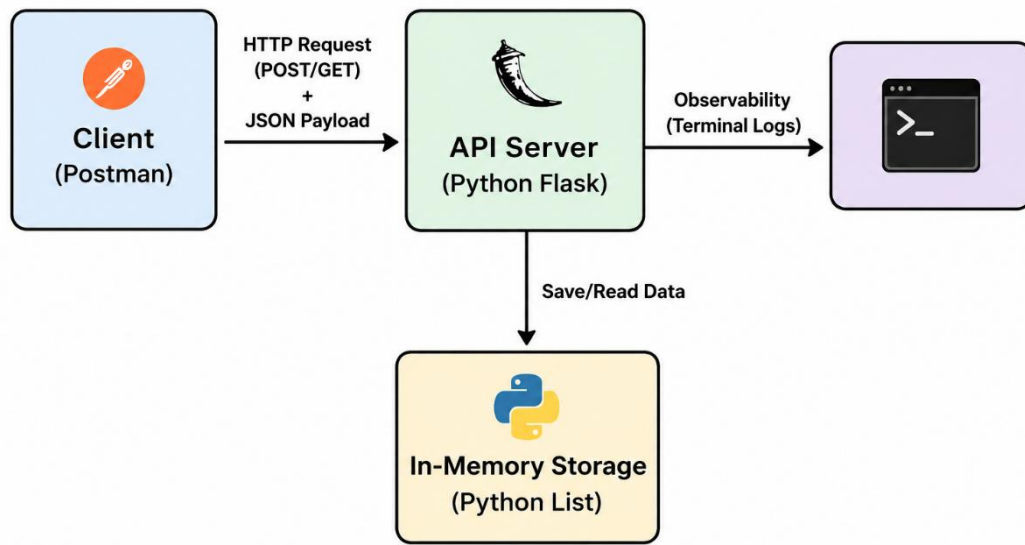
1.3 Alasan Memilih Case

Case **API Ingestion Dataset** berbasis REST dan JSON dipilih karena merepresentasikan kebutuhan nyata di industri data saat ini. Sebagai mahasiswa Sains Data, memahami bagaimana data mengalir secara aman dan terstruktur melalui jaringan komputer merupakan kompetensi dasar yang vital untuk membangun arsitektur data (*data engineering*) yang solid

BAB 2

FLOW DIAGRAM

2.1 Diagram komponen



2.2 Penjelasan Data Flow

1. **Request Initiation:** *Client* (Postman) menginisiasi komunikasi dengan mengirimkan paket *HTTP Request* ke alamat server lokal `http://localhost:5000`. *Request* membawa instruksi metode tertentu (GET, POST, atau DELETE) beserta data berformat JSON jika menggunakan metode POST.
2. **Processing & Validation:** Server API berbasis Python Flask menerima paket data. Server mengekstrak payload, membuat *Request ID* unik via modul UUID, dan menjalankan fungsi validasi kolom data.
3. **Observability Logging:** Server secara instan mencetak baris log ke konsol terminal lokal berisi *timestamp*, *Request ID*, metode HTTP, *endpoint* tujuan, dan status kode respons.
4. **State Persistence:** Jika data valid (POST), data dimasukkan ke dalam objek memori RAM server. Jika permintaan berupa pengambilan data (GET), server mengambil data dari objek tersebut.
5. **Response Delivery:** Server mengemas data balik atau pesan *error* ke dalam struktur JSON terstandarisasi, lalu mengirimkannya kembali ke Postman bersama dengan HTTP Status Code yang sesuai.

BAB 3

SELEKSI PROTOKOL

3.1 Alasan memilih protocol REST API over HTTP/1.1

Project ini mengintegrasikan protocol REST API berbasis HTTP/1.1 dengan pertukaran data berformat JSON. Memilih ini berdasarkan pada keunggulan beberapa teknis sebagai berikut :

- **Sifat Stateless:** Setiap interaksi request bersifat mandiri. Server tidak perlu menyimpan status sesi *client*, sehingga meringankan beban kerja server dan membuat arsitektur jaringan menjadi sangat terukur (*scalable*).
- **Kesesuaian Format JSON untuk Sains Data:** Objek JSON (*JavaScript Object Notation*) memiliki struktur pasangan *key-value* yang secara alami sangat selaras dengan tipe data Dictionary pada Python. Hal ini memudahkan konversi data mentah dari jaringan langsung menjadi objek DataFrame (Pandas) untuk keperluan analisis lanjut.
- **Semantik Method yang Jelas:** REST menggunakan metode bawaan HTTP yang mencerminkan aksi manipulasi data secara presisi (GET untuk membaca, POST untuk membuat/ingest, dan DELETE untuk menghapus)

BAB 4

ENDPOINT DAN AKSI PROTOKOL

RANCANGAN ARSITEKTUR KONTRAK ANTARMUKA API

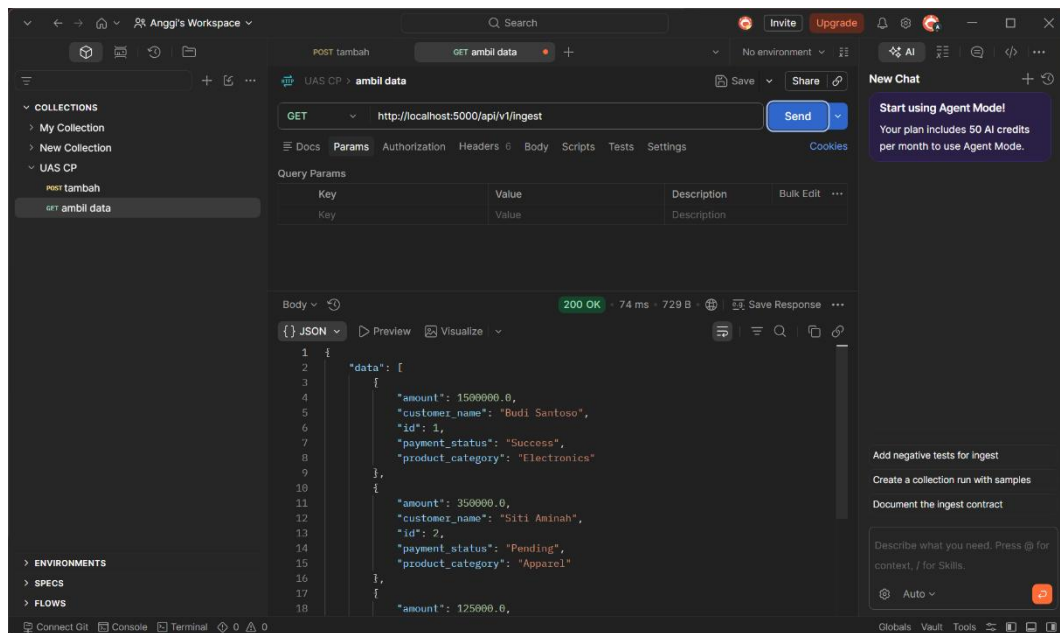
Sistem Server Ingestion

HTTP Method	Endpoint (Path/URL)	Fungsi Utama	Request Body	Expected Status Code
GET 	/api/v1/health	Memeriksa apakah server API berjalan dengan aktif.	Tidak ada (Empty)	 200 OK
GET 	/api/v1/ingest	Menarik seluruh dataset transaksi yang tersimpan di memori.	Tidak ada (Empty)	 200 OK
GET 	/api/v1/ingest/<id>	Mengambil baris data transaksi spesifik berdasarkan nomor ID.	Tidak ada (Empty)	 200 OK  404 Not Found
POST 	/api/v1/ingest	Melakukan ingest (penyerapan) data transaksi baru ke sistem.	JSON berisi: - customer_name - product_category - amount - payment_status	 201 Created  400 Bad Request
DELETE 	/api/v1/ingest/<id>	Menghapus baris data transaksi tertentu dari memori server.	Tidak ada (Empty)	 200 OK  404 Not Found

BAB 5

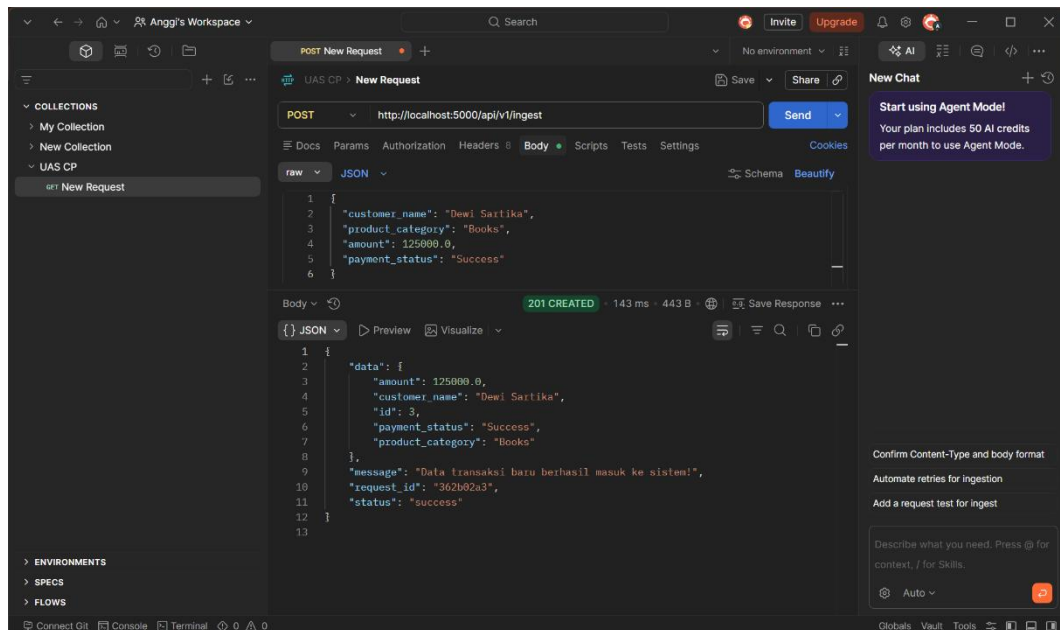
BUKTI PENGUJIAN TEKNIS

5.1 Penarikan seluruh baris data (GEL ALL)



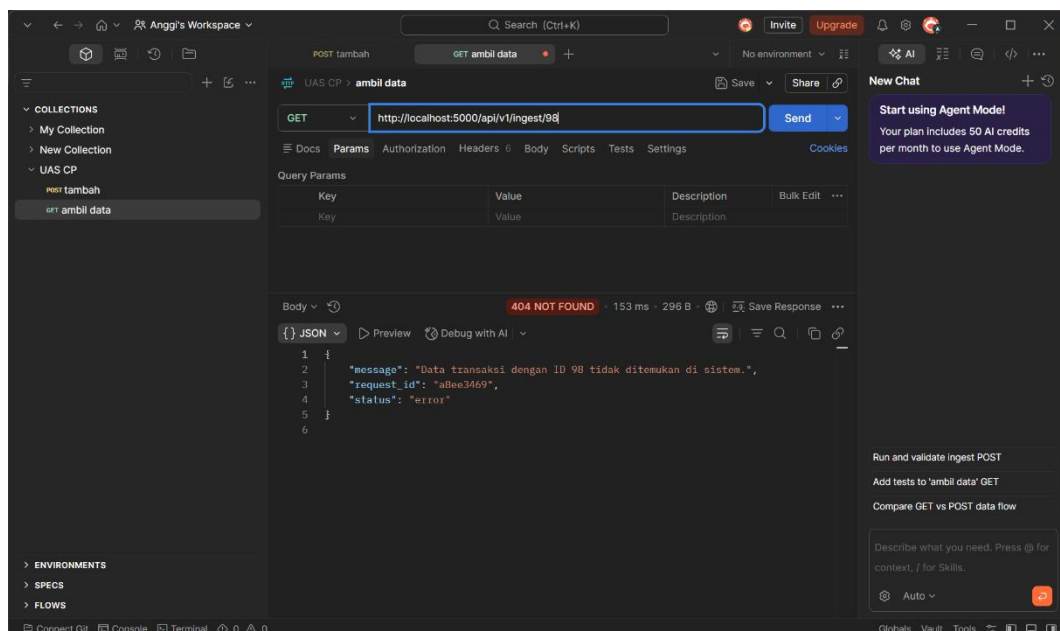
Analisis : Request diarahkan ke `/api/v1/ingest` dengan metode GET. Aplikasi berhasil membalas dengan HTTP Status Code 200 OK dan menampilkan muatan struktur data mentah bawaan sistem secara lengkap dalam format array JSON.

5.2 Ingestion Data Transaksi baru (POST Created)



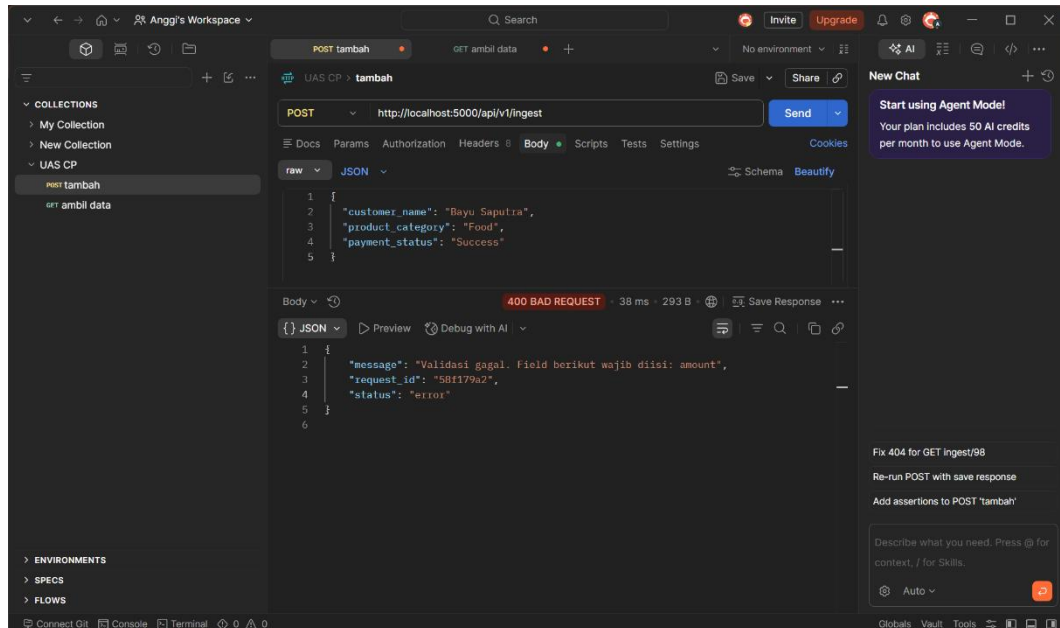
Analisis : *Client* mengirimkan data transaksi baru berformat JSON dengan atribut lengkap. Sistem berhasil melakukan ekstraksi, memberikan validasi hijau, menyimpan data ke RAM, dan mengembalikan HTTP Status Code 201 Created.

5.3 Akses sumber data tidak terdaftar (GET 404)



Analisis : Pengujian dilakukan dengan menembak data ID yang salah (`/api/v1/ingest/98`). Server memberikan proteksi keamanan jaringan dengan mengembalikan HTTP Status Code 404 Not Found, mencegah penyerapan data kosong oleh *client*.

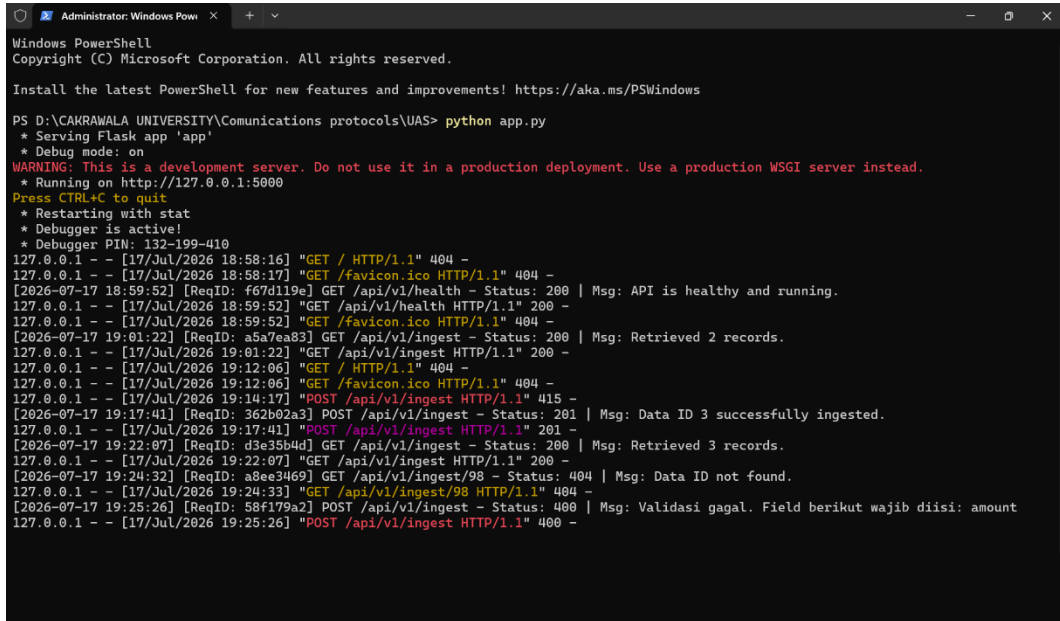
5.4 Pengiriman data cacat (POST 400)



Analisis : *Client* sengaja mengirimkan payload tanpa menyertakan field wajib `amount`. Fungsi logika server berhasil mendeteksi kecacatan skema data tersebut, memblokir interaksi jaringan, dan mengirimkan respons HTTP Status Code 400 Bad Request beserta pesan galat penjelasan field yang hilang

BAB 6

BUKTI OBSERVABILITAS dan ANALISIS TRAFIK



```
Administrator: Windows Powe...
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

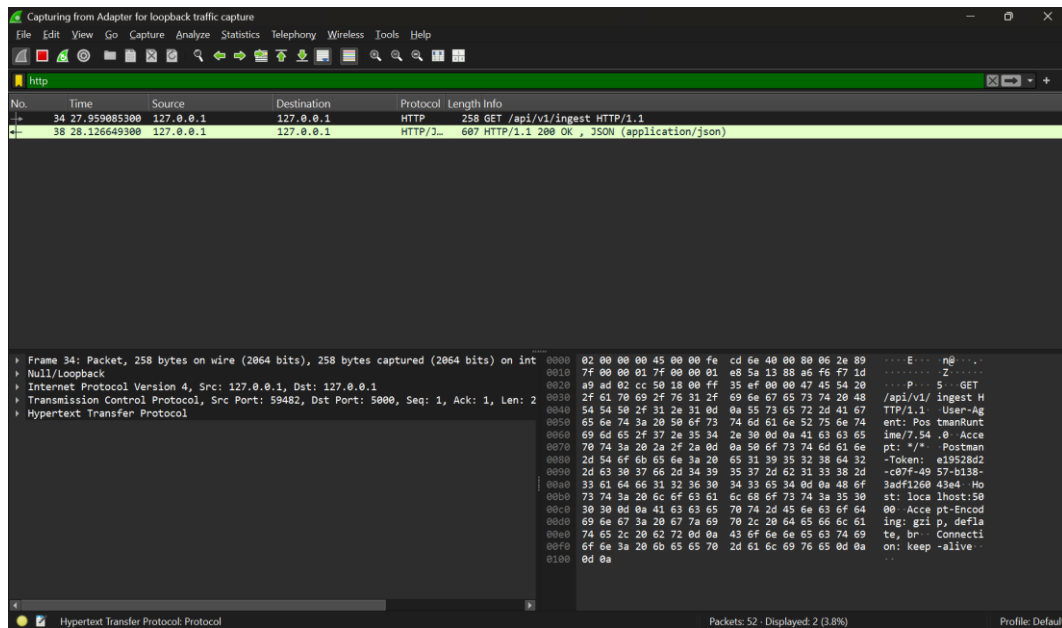
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\CAKRAWALA UNIVERSITY\Communications protocols\UAS> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 132-199-410
127.0.0.1 - - [17/Jul/2026 18:58:16] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [17/Jul/2026 18:58:17] "GET /favicon.ico HTTP/1.1" 404 -
[2026-07-17 18:59:52] [ReqID: f67d119e] GET /api/v1/health - Status: 200 | Msg: API is healthy and running.
127.0.0.1 - - [17/Jul/2026 18:59:52] "GET /api/v1/health HTTP/1.1" 200 -
127.0.0.1 - - [17/Jul/2026 18:59:52] "GET /favicon.ico HTTP/1.1" 404 -
[2026-07-17 19:01:22] [ReqID: a5a7ea83] GET /api/v1/ingest - Status: 200 | Msg: Retrieved 2 records.
127.0.0.1 - - [17/Jul/2026 19:01:22] "GET /api/v1/ingest HTTP/1.1" 200 -
127.0.0.1 - - [17/Jul/2026 19:12:06] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [17/Jul/2026 19:12:06] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [17/Jul/2026 19:14:17] "POST /api/v1/ingest HTTP/1.1" 415 -
[2026-07-17 19:17:41] [ReqID: 362b02a3] POST /api/v1/ingest - Status: 201 | Msg: Data ID 3 successfully ingested.
127.0.0.1 - - [17/Jul/2026 19:17:41] "POST /api/v1/ingest HTTP/1.1" 201 -
[2026-07-17 19:22:07] [ReqID: d3e35b4d] GET /api/v1/ingest - Status: 200 | Msg: Retrieved 3 records.
127.0.0.1 - - [17/Jul/2026 19:22:07] "GET /api/v1/ingest HTTP/1.1" 200 -
[2026-07-17 19:24:32] [ReqID: a8ee3469] GET /api/v1/ingest/98 - Status: 404 | Msg: Data ID not found.
127.0.0.1 - - [17/Jul/2026 19:24:33] "GET /api/v1/ingest/98 HTTP/1.1" 404 -
[2026-07-17 19:25:26] [ReqID: 58f179a2] POST /api/v1/ingest - Status: 400 | Msg: Validasi gagal. Field berikut wajib diisi: amount
127.0.0.1 - - [17/Jul/2026 19:25:26] "POST /api/v1/ingest HTTP/1.1" 400 -
```

Analisis : Aplikasi mengimplementasikan mekanisme *Application Logging* mandiri pada konsol server. Setiap baris log merekam informasi kritis untuk audit:

- *Timestamp*: Mencatat waktu presisi kapan request menyentuh server.
- *Correlation ID (Request ID)*: String acak unik sepanjang 8 karakter berbasis UUIDv4 diproduksi otomatis per interaksi. ID ini berfungsi untuk melacak jejak perjalanan satu paket data spesifik di dalam jaringan dari hulu ke hilir jika terjadi gangguan operasional.
- *Method & Endpoint*: Mengidentifikasi jenis aksi protokol komunikasi.

Analisis trafik WireShark dan Limitasi jaringan



Proses pengembangan dan pengujian mini project ini dijalankan sepenuhnya pada lingkungan lokal menggunakan alamat *loopback* IP (127.0.0.1 atau localhost). Terdapat limitasi teknis bawaan pada sistem operasi di mana paket data internal lokal tidak dialirkan melewati kartu antarmuka jaringan fisik (*Network Interface Card* seperti Wi-Fi atau Ethernet adapter). Akibatnya, aplikasi pengendus paket standar seperti Wireshark tidak dapat menangkap (*capture*) paket data HTTP tersebut tanpa instalasi adapter loopback khusus (seperti Npcap loopback driver). Sebagai solusi mitigasi kepatuhan terhadap aspek observabilitas, pemantauan dialihkan secara penuh menggunakan *Application Server Logs* berbasis konsol terminal yang terbukti memberikan informasi status kode dan pelacakan request secara presisi.

BAB 7

ANALISIS KEANDALAN PROTOKOL

7.1 Mekanisme Error

Keandalan (*reliability*) sebuah protokol komunikasi dinilai dari bagaimana sistem merespons skenario kegagalan tanpa menyebabkan server mengalami mati total (*application crash*). Pada mini project ini, *Error Contract* dibangun menggunakan standardisasi respons JSON terpadu. Ketika terjadi kegagalan (seperti 400 Bad Request atau 404 Not Found), sistem akan membungkus informasi galat ke dalam objek dengan kunci status: error dan memberikan pesan penjelasan yang aman tanpa membocorkan baris kode internal aplikasi.

7.2 Pencegahan Kerusakan Pipa Data

Dengan memblokir payload yang tidak lengkap di gerbang API (melalui pengecekan status *required fields*), server menjamin bahwa hanya data yang 100% bersih dan valid yang dapat masuk ke repositori penyimpanan. Bagi seorang praktisi Sains Data, keandalan penapisan jaringan ini menghemat waktu komputasi pembersihan data (*data cleaning*) secara signifikan di tahap pasca-produksi

BAB 8

KESIMPULAN

Proyek pengembangan *API Ingestion Dataset* menggunakan arsitektur REST API dan bahasa Python Flask telah berhasil diimplementasikan dengan sukses penuh. Sistem mampu memproses 5 variasi endpoint tindakan protokol komunikasi, menyaring gangguan input kotor melalui pembatasan HTTP Status Code 400 dan 404, serta mendokumentasikan setiap riwayat transaksi jaringan secara transparan menggunakan *Request ID* unik.

8.1 Batasan Sistem (Limitation)

- **Volatilitas Penyimpanan:** Data transaksi disimpan pada memori volatile RAM internal Python (*In-Memory Database*). Jika proses server dihentikan atau komputer dimatikan, seluruh data transaksi baru yang telah diserap akan hilang kembali ke kondisi semula karena belum terintegrasi dengan media penyimpanan database persisten.
- **Aspek Keamanan Jaringan:** Akses API masih terbuka bebas tanpa adanya sistem otentikasi pertukaran token (seperti API Key atau JWT).

8.2 Rencana Pengembangan Selanjutnya (Improvement)

- Mengintegrasikan gerbang API Flask dengan sistem database relasional fisik (seperti PostgreSQL atau SQLite) agar data tersimpan secara permanen.
- Menambahkan lapisan keamanan enkripsi TLS/HTTPS dan otentikasi *Header* berupa Authorization: Bearer <token> untuk melindungi data transaksi dari intersepsi pihak asing di jaringan luar.

8.3 Refleksi Pembelajaran

Melalui pengerjaan mini project UAS secara mandiri ini, penulis mendapatkan pemahaman mendalam secara praktis mengenai konsep teoretis mata kuliah *Communication Protocol*[cite: 2]. Penulis menyadari bahwa performa keandalan sebuah sistem sains data tidak hanya dinilai dari algoritma pemodelannya, melainkan sangat bergantung pada kualitas rancangan protokol komunikasi data yang menghubungkan setiap komponen di dalam jaringan komputer